

ECE532: Digital Systems Design
Final Project Group Report
April 7, 2026

Real-time 2D Motion Capture System

Group 9:
Tiger Luo
Cary Wu
Ethan Luan
Stephen Lee

Table of Contents

1. Overview	3
1.1 Background	3
1.2 Motivation	3
1.3 Goals	3
1.4 Original proposed Block Diagram	3
1.5 System Changes	4
1.6 Brief Descriptions of the IP	5
2. Outcome	6
2.1 Results	6
2.2 Future Work	7
3. Project Schedule	8
3.1 Projected vs. Delivered Milestones	8
3.2 Evaluation of Milestone Progress and Differences	9
4. Detailed Description of the IP blocks	10
4.1 Video Data Path	10
4.1.1 HDMI Input/Output	10
4.1.2 Video/AXIS Conversion	10
4.1.3 Video Timing	10
4.2 Custom IP	11
4.2.1 Top-Level	12
4.2.2 Joint Detection	12
4.2.2.1 Pixel Detection	12
4.2.2.2 Edge Detection	13
4.2.2.3 Circle Tracking	13
4.3 Microblaze Processor + AXI peripherals	14
4.3.1 Microblaze Processor	14
4.3.2 AXI UART	16
4.3.3 AXI GPIO	16
5. Design Tree Description/Video Link	18
6. Advice for Future Students	19
References	20

1. Overview

1.1 Background

Motion capture is the process of digitally tracking and recording movement patterns by identifying specific points on a subject over time. It is widely used in a variety of applications including animation, film, video games, and sports analysis, where accurate movement is essential for realistic visualization and analysis. There are a variety of methods to do motion capture involving hardware such as camera arrays, reflective markers, or inertial sensors.

1.2 Motivation

The team's motivation to work on this project was that we were interested in developing a fun and interactive design, especially something based on something commonly used in modern multimedia systems. Traditional motion capture systems rely on specialized hardware and while they typically provide high accuracy, they are also expensive and complex. In contrast, approaches using standard video inputs and real-time processing offer a more accessible alternative.

1.3 Goals

The primary goal of this project is to develop a real-time 2D motion capture system. The system would capture a 1280×720 HDMI input stream, process each frame, draw output frames, and output via HDMI with minimal latency at 60FPS. Coloured markers are placed on an individual to denote specific joint locations, enabling their detection within each frame. This is accomplished by developing a design on an FPGA capable of operating high-throughput pixel data using techniques discussed in later sections, along with a custom IP performing marker detection on a streaming input. The detected marker data is then processed by a MicroBlaze processor, which interprets the joint positions and classifies them based on colour into elbow, shoulder, etc, in order to construct a skeletal representation of the subject. Finally, the processed video stream is output to an external monitor. The system is expected to operate continuously on live video input while maintaining stable performance and producing accurate, real-time visualizations of motion.

1.4 Original proposed Block Diagram

Shown in Figure 1 is our original system block diagram. In this iteration, the idea was to have a single pipeline where input images entered through a camera are processed via a custom IP to detect markers, which are then given to the Microblaze processor. The Microblaze processor then takes marker locations, draws an output frame based on the given markers, and the output frame is then given to the HDMI out system and displayed to a screen. The original intention of this architecture is to separate the dataflow into an input-side and output-side that don't use the same resources. As a stretch goal, we wished to load images onto an SD-card that could be drawn over the output skeleton, analogous to how models are rigged in a 3D motion capture system.

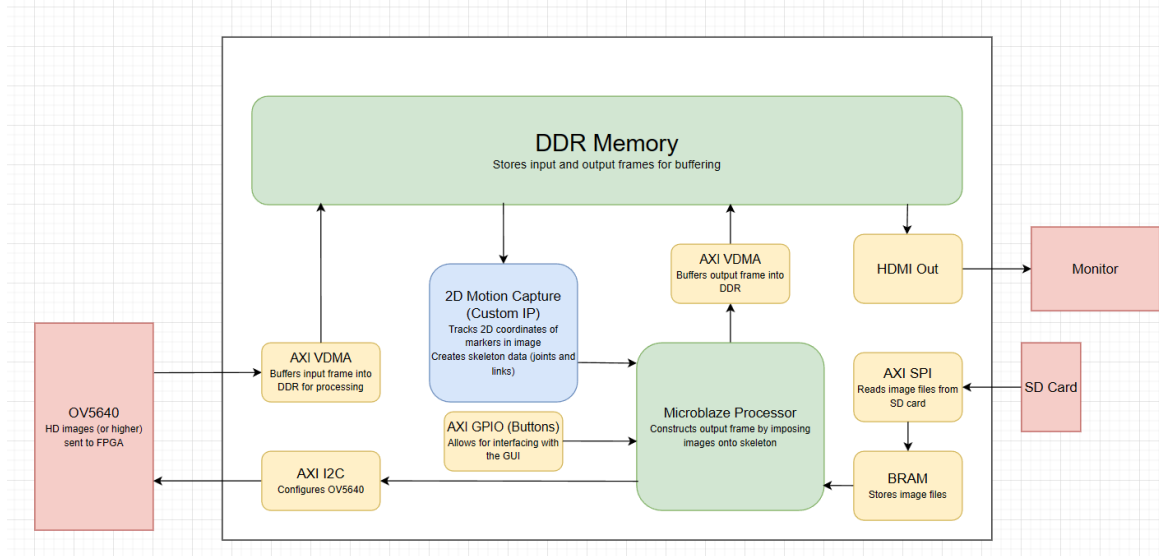


Figure 1. Original system block diagram.

1.5 System Changes

In our final design, instead of splitting the system into an input and output-side, it was split 3 ways: Video stream, custom IP, and Microblaze processor. The best way of picturing the system is through the three modules capable of accessing framebuffers in memory: The video dedicated VDMA, the custom IP dedicated VDMA, and the Microblaze processor. Input and output video streams are connected in a single pipeline, where the video dedicated VDMA is constantly reading in and out frame data from framebuffer locations without regard for the other two sides. The custom IP dedicated VDMA reads input framebuffer to the custom IP constantly, while the Microblaze processor uses joint locations to draw on the output framebuffer. Note that the custom IP processing and the Microblaze output drawing happen serially, not in parallel, meaning output FPS is band limited by the Microblaze drawing speed. Synchronizations and a double buffer are facilitated through the use of VDMA interrupt events.

Additionally, there are two changed modules: First, we couldn't reach our stretch goal of adding an SD card. Second, the camera used didn't function with the PMOD headers JB and JC due their pins not properly using IO format LVCMOS33 (exact reason is unknown, but is probably due to the differential routing) [1]. As a result, the camera was replaced with an HDMI input instead to maintain complexity score. Our HDMI source was a group member's laptop set to extend screen mode in 1280x720p. The remaining additional blocks (DVI2RGB, VTC, and Video to AXIS) are a result of following Digilent's HDMI tutorial to learn how HDMI functions, and incorporating their IP blocks in our design [2].

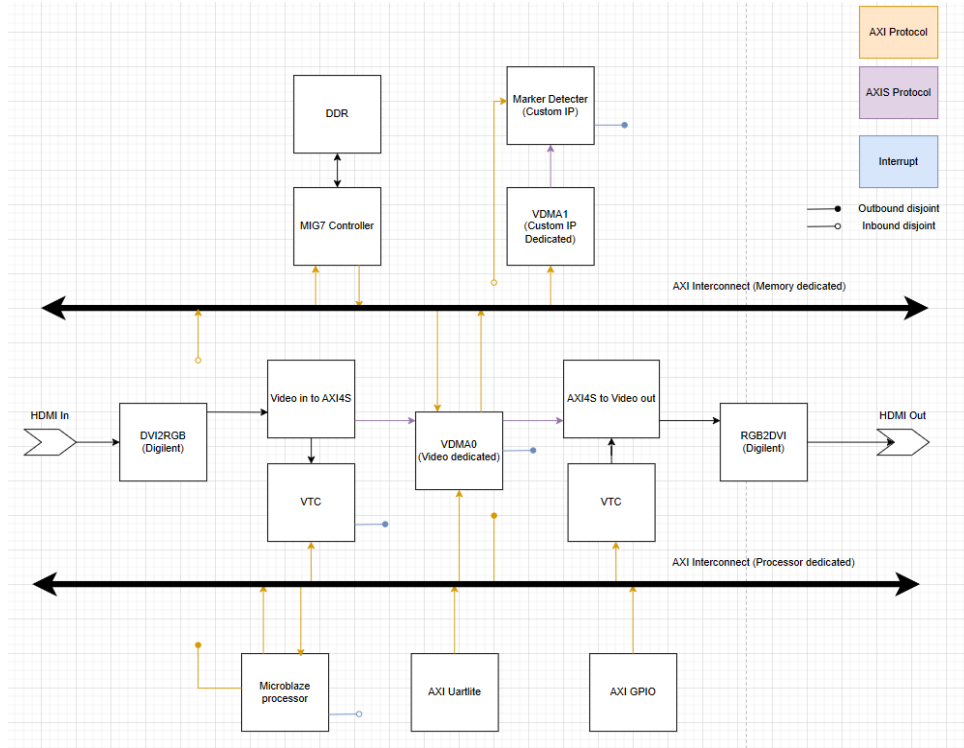


Figure 2. Final block diagram (There are disjointed connectors for better readability).

1.6 Brief Descriptions of the IP

IP Block	Description
DVI2RGB/RGB2DVI	Digilent IP. Convert TMDS to clocked video signal, a 24 bit RGB format with video synchronization signals (vsync, hsync, etc) [3]
Video to AXI4S / AXI4S to Video	Vivado IP. Convert video signal to AXIS data and video synchronization signals as a separate channel or vice versa.
VTC	Vivado IP. Detect video resolution on the input side, used to generate video synchronization signals based on resolution on output side.
VDMA	Vivado IP. Store or read data from one of three framebuffers in memory. A video dedicated VDMA handles both HDMI input and output, and a custom IP dedicated VDMA handles reading to custom IP only.
MIG7 Controller	Vivado IP. Interface with DDR memory. 3 framebuffers at 1280x720p is around 8.1MB, necessitating its use.
Microblaze	Vivado IP. Configure all IPs and draw output frames directly to memory.
Uartlite + GPIO	Vivado IP. Interface with board at runtime
Marker detector	Custom IP. Takes in an AXIS and processes a frame pixel by pixel, outputs marker locations to slave registers and sends an interrupt when done.

Table 1: List and description of IPs.

2. Outcome

Overall the project successfully detects markers and implements the video pipeline to process and produce smooth, 30 FPS output at 1280x720p. However, colour detection, which is used to classify joints, is still noisy and can be inaccurate due to lighting conditions. It should be noted that to make the system work, some setup was required. An all black background was placed behind the model, and some pre-processing on the input video was done with the use of OBS to make colours more saturated.

2.1 Results

Our system can be configured through the use of GPIO buttons into multiple modes: colour or B/W, video or photo mode. Regarding video streaming, our custom IP and Microblaze drawing can process and create up to 30 FPS. Output video is smooth without flickering or stuttering, as once the system is set to video mode, the only variance in timing would be in HDMI IO and DRAM read times. The GUI displays FPS and current operating mode: video/photo (left/right), coloured/white toggle (bottom), stop (middle), and exit (top).

Our marker detector custom IP is accurate at detecting white or coloured markers, but only under specific circumstances. The markers cannot be too large or small. Markers must be at least 10 pixels in height to count as a marker. Maximum size wasn't factored into our algorithm, but in general if a marker is greater than around 150 pixels, it will begin to detect it as two markers due to black noise (See section 4.2.x for more details). The system can reliably denoise white markers of appropriate size, meaning no duplicates and no random noise being picked up as markers. However, depending on the setup, the denoising parameters may need to be adjusted.

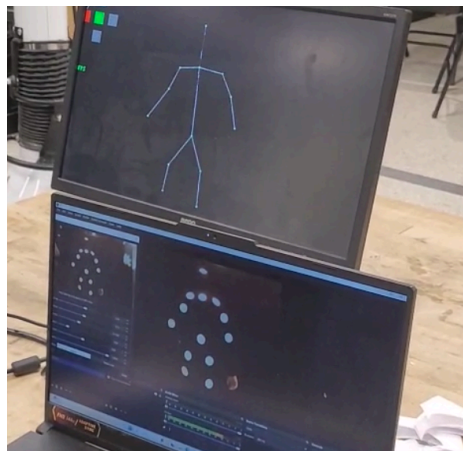


Figure 3. White marker detection with model

Coloured marker detection was less reliable. There were two major issues, the first was that depending on the lighting and the angle, markers will reflect a lot of light and can appear as white since paper is relatively glossy. The second issue was that our method of detecting colours through RGB thresholds was more difficult to distinguish colours. Most coloured paper is relatively desaturated, so its RGB values tend to be lower and more normalized among its relative values (See Figure 4). We could only reliably detect three colours using RGB thresholds: Red, green, and yellow. However, due to the first

issue, yellow couldn't be detected since it was too reflective. The green coloured markers were problematic due to its reflectivity almost making it unusable, and had to be angled downwards away from the light. Under the right conditions, we were able to incorporate three marker colours successfully: Green, red, and white, which provided enough variety to distinguish limbs from the torso.

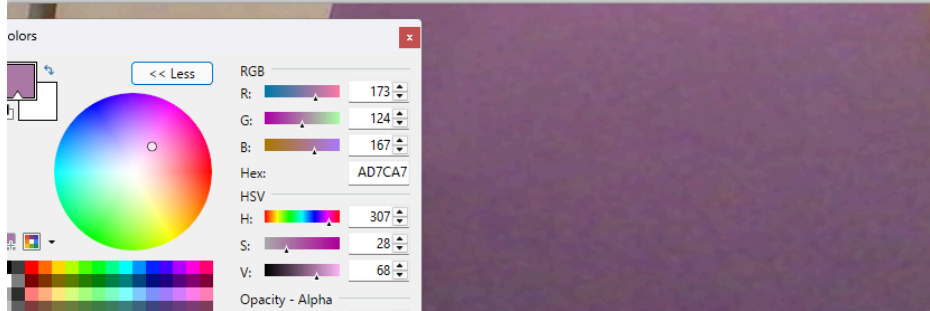


Figure 4. RGB values of purple coloured paper.

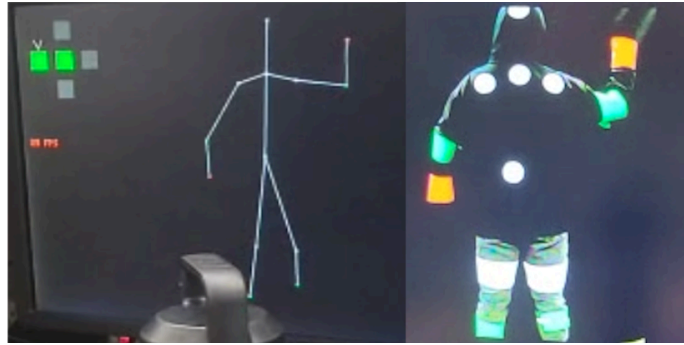


Figure 5. Coloured marker detection. Note that green markers are close to appearing white.

Regarding timing, there is a timing violation within the custom IP but this does not break it. It involves an intra-clock path violation internally, where one of the long-term logic variables violates the path to the output. This data transfer only occurs once at the end of the frame, not every clock cycle, and is held at the value for multiple clock cycles both prior and after the end of frame. Thus no real timing issues occurred.

2.2 Future Work

Regarding FPS and video processing, while we were able to achieve a high FPS, this was only because the output frame drawing requires a small number of pixels ranging in the hundreds to low thousands. If it required a larger number of pixels, the FPS would drop dramatically because the Microblaze processor would require much more time to draw the frame. This would have bottlenecked our original goal of rigging a model to the skeleton. To get around this, another custom IP would be needed to draw the output frame.

To resolve the coloured marker issue, one way would be to get better markers that are matte and more saturated. This would make the RGB values more distinguishable and easier to use. However, a cheaper and more accessible alternative would be to switch to HSV colour format. Using this method, H values can be used to determine colour by bounding colour definitions within an angle, while V can be used to detect white and black.

3. Project Schedule

3.1 Projected vs. Delivered Milestones

Our team's project milestones provided a framework for developing the motion-capture system. A comparison between our projected milestones and delivered milestones are present in Table 2.

Milestone	Original Milestones	Actual Milestones
1	• Write off-board C program to exemplify skeleton creation • Complete block diagram • Research motion capture setup	• Wrote off-board C program to exemplify skeleton creation • Connected majority of Vivado IP blocks needed for the project • Followed Diligent tutorial to implement HDMI on Nexys Video Board
2	• Write on-board C program to be run on Microblaze processor • Integrate OV5640, VDMA, DDR, and HDMI for complete video stream • Obtain materials for motion capture setup	• Set up Verilog code and testbench for custom IP • Created block diagram for OV5640 to HDMI video stream • Integrated HDMI systems and partially integrated custom IP
3	• Integrate OV5640, custom IP, and Microblaze for a single image	• Custom IP and skeleton generation code tested and integrated in simulation • OV5640 input verification stalled because camera was broken • Verified HDMI output
4	• Optimize flow and improve timing and synchronization to achieve 60 FPS and real-time processing	• Custom IP integrated with external HDMI input and output
5	• Perform testing and ensure reliability of the system	• Pipelined and implemented denoising on custom IP • Integrated custom IP with video stream • Created GUI to switch modes and control system actions
6	• Present completed motion capture system • Camera captures real-time footage and output skeleton is shown on GUI	• Implemented colour detection on custom IP • Marker detection working with real video • Created joint classification logic

Table 2: Original and delivered project milestones.

3.2 Evaluation of Milestone Progress and Differences

Throughout the project, the team faced challenges that shifted the timeline from the projected milestones and altered priorities and various stages. The most significant difference between the original and actual milestones was that the team planned to have the custom IP integrated and the system working for a single still image by milestone 3 and the Mid-Project Demo. However, it was not until milestone 5 that the team had fully integrated the entire system and achieved this functionality.

There were several reasons for the delay regarding integrating the system. Firstly, the team initially wanted to use the OV5640 camera module for video input. However, after creating the block diagram for video streaming and undergoing initial testing, it was revealed that the provided camera was broken. The internal silicon chip was burned and one of the pins was internally shorted. Furthermore, after replacing the module the OV5640 did not function with the Nexys Video Board PMOD headers. Afterwards, the team replaced the module with HDMI input. However, this resulted in a general delay of the system.

Secondly, the team took additional time to integrate the system due to a variety of challenges including, synthesis problems from memory address inconsistencies, bugs with specific Vivado versions, timing issues with the custom IP being too slow, and custom IP synchronization with software. Gradually, these issues were debugged and resolved through various methods including using different Vivado versions, ILAs and simulations, adding interrupts, and pipelining the custom IP to improve timing. While most of these problems were fixed and a working design was created, this also added to the delay in the timeline.

Other differences between the original goal and actual milestone were that the team initially wanted to obtain 60 FPS on the system. However, primarily due to the timing constraints of the custom IP, which had already been improved using pipelining, this was ultimately decreased to 30 FPS. Lastly, the team planned to research and obtain materials for the physical motion capture setup early in the project such that it could be ready for physical testing. However, due to other delays in the development of the system itself this was decreased in priority until later in the project.

4. Detailed Description of the IP blocks

4.1 Video Data Path

4.1.1 HDMI Input/Output

HDMI input and output is handled by the DVI2RGB and RGB2DVI blocks respectively, which are both IP from Digilent's IP library [3]. On the input side the DVI2RGB converts differentially signalled TMDS from HDMI input into clocked outputs. It begins by recovering timing directly from the incoming TMDS clock channel, generating both a high-speed serial clock and a lower-frequency pixel clock with a fixed 5:1 ratio. Using these clocks, each of the three TMDS data channels (corresponding to red, green, and blue) is deserialized and dynamically phase-aligned. The output is a 24-bit bus in RBG format (not a typo, it is RBG and not RGB format), accompanied by standard video timing signals (horizontal sync, vertical sync, and video active) and a recovered pixel clock. Additionally, the core can emulate a display device through an EDID ROM exposed over the DDC interface, allowing upstream video sources to automatically select compatible resolutions. In our case, we force the source to comply with 1280x720p by making it the only one available.

On the output side, the RGB2DVI does the inverse operation. It takes a 24 bit RGB bus with standard video timing signals, a pixel clock, and a higher frequency serial clock (same 5:1 ratio) to generate output TMDS differential signals that are output to the HDMI interface. Both IP's source code can be found here: <https://github.com/Digilent/vivado-library/tree/master/ip>.

4.1.2 Video/AXIS Conversion

On the input side, the 24 bit RGB bus with video timing signals are fed to a Video-In to AXI4-Stream block from the standard Vivado library. It takes the recovered pixel clock from the DVI2RGB block as the input clock. It then converts it into two outputs, a 24-bit width AXI4S channel with handshake signals, and a video timing channel with hsync, V-sync, and video active to be fed to the VTC. The AXI4S is routed to a Vivado Standard VDMA, which stores the input into one of three framebuffers.

That same VDMA is responsible for outputting the data from one of the three framebuffers to an AXI4-Stream to Video Out block, while the video timing for the block is generated by a separate VTC. The AXI4-Stream to Video Out block then packages the input data and video timing signals into an output video signal that gets fed to the RGB2DVI block.

4.1.3 Video Timing

The Video Timing Controller (VTC) is used to detect and set video resolution and synchronization signals. On the input side, the VTC is configured as a detector only, where Video-In to AXI4-Stream block creates a `vtiming_out` signal for the input VTC to detect. Only 3 signals are detected: V-sync for when a frame is done, hsync for when a line is done, and video active for when video is being transmitted. It uses these signals to detect input video resolution and timing structure based on the timing between each sync signal. This information is used to set the VDMA's horizontal and vertical size. Interrupts are only raised when a change in timing is detected to update resolution.

On the output side, a VTC is set to generator mode, using internal clocks and configured through AXI to the desired resolution, and creates the output timing signals vsync, hsync, and active video. These timing signals are then given to the AXI4-Stream to Video Out block, and are bundled with the data from the VDMA to a video output stream.

The clock tree for the entire pipeline is as follows: All AXI interfaces use the same 100MHz clock generated by the MIG7. The recovered pixel clock from the DVI2RGB block is used for clocked RGB data transfer between DVI2RGB to Video In to AXI4-Stream blocks, as well as the input VTC's reference clock. In order to create the output pixel and serial clock at a 5:1 ratio for the RGB2DVI block, the Dynamic Clock Generator (dynclk) Digilent IP is used. This acts as a MMCM with dividers that can be adjusted at runtime, which is used to adjust the output clock frequency based on the resolution [4]. The created pixel clock is also used for the video-dedicated VDMA MM2S channel, the output VTC reference clock, and the video stream of the AXI4-Stream to Video Out block. The source code for this IP can be found here: <https://github.com/Digilent/vivado-library/tree/master/ip>.

4.2 Custom IP

The Custom IP implements the detection of the input frame's joint markers, which are then used by the Microblaze Processor in drawing the output frame skeleton. A simplified high-level block diagram is shown in Figure 6. The top-level file of the IP is titled **custom_ip**, instantiating the core logic of the design in **joint_detect** and handling the communication both upstream via AXI-Stream protocol and downstream via AXI4-lite protocol.

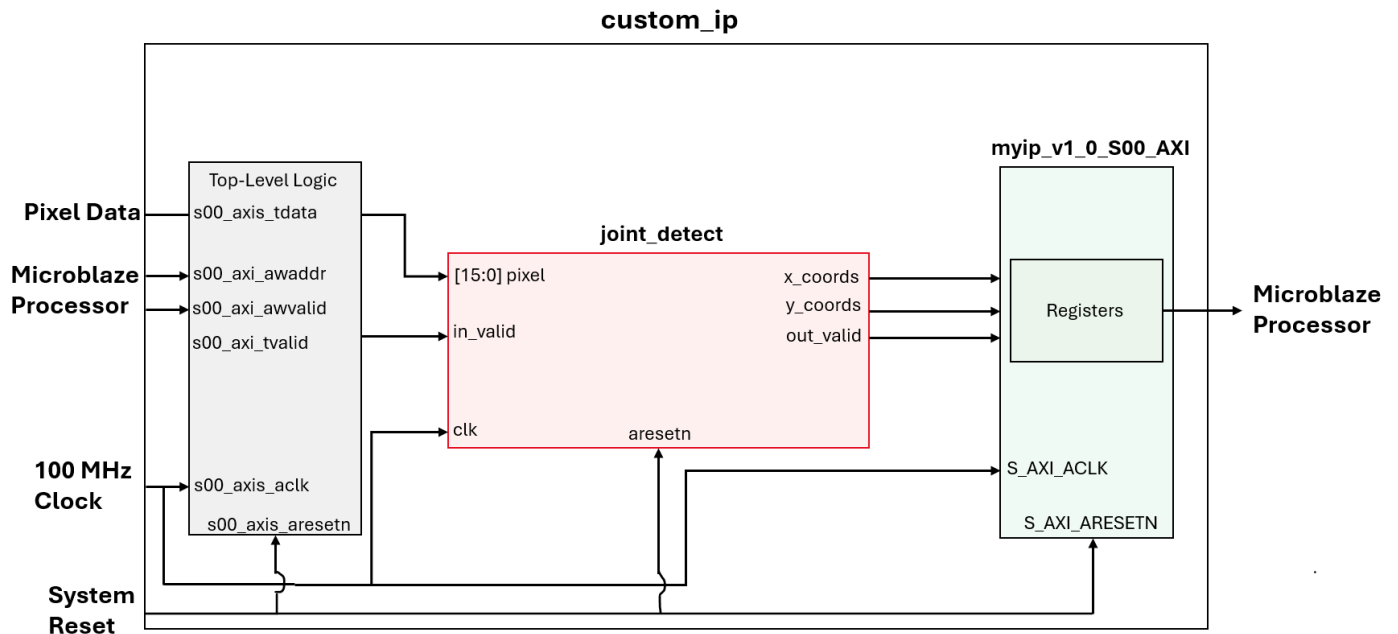


Figure 6. High-level block diagram of the Custom IP.

4.2.1 Top-Level

The Custom IP takes in a continuous stream of pixels in 24-bit RGB888 format sent at the system's clock speed, 100 MHz. The incoming pixels are converted to a RGB565 format simply by shifting the corresponding LSBs out for each R, G, and B portion. Using AXI-Stream protocol, the **joint_detect** module will only begin processing these pixels when *in_valid* is set high by two signals, *s00_axis_tready* and *s00_axis_tvalid*. The signal *s00_axis_tready* is set high when the Microblaze Processor writes anything to address 15 and indicates whether the Custom IP is activated or not. The signal *s00_axis_tvalid* is received upstream along with the input pixel data.

When the **joint_detect** module completes the processing of a frame, it sends the x and y coordinates of the centers of each marker along with a valid signal to the internal registers of module **myip_v1_0_S00_AXI**. This module is a Vivado-generated AXI4-lite slave interface, with unchanged read functionality and a modified write functionality. The output coordinates of the **joint_detect** module and the corresponding valid signal are written into slave registers *slv_reg0* to *slv_reg12* and *slv_reg15* respectively every clock cycle. Then in order for the Microblaze Processor to read the output, it initiates a read request using AXI4-lite protocol to obtain these coordinates and also determine if these coordinates are valid.

4.2.2 Joint Detection

The **joint_detect** module implements marker detection using a three-stage, in-order pipeline. The first stage converts the incoming 24-bit RGB data into a single-bit black or white value. Then, the next stage determines any transitions in this black-and-white frame to track segments of markers. Finally, the last stage computes whether these segments are part of a marker already being tracked, are part of a new marker, or are not part of a marker at all.

4.2.2.1 Pixel Detection

In the first stage of the pipeline, "Pixel Detection", the pixel is first checked for validity using the AXI-Stream protocol mentioned above. Given a valid pixel, the module compares its RGB data against a set of colour thresholds, *THRESHOLD_R*, *THRESHOLD_G*, and *THRESHOLD_B*, to determine whether the pixel lies in a range of any of the specified marker colours. The chosen marker colours to be used were red, green, and white due to a multitude of factors. Firstly, only three colours were needed to demonstrate the functionality of our design due to the constraints placed on the 2D skeleton's movement which is further explained in Section 4.3.1. Furthermore, it was discovered that red, white, and green were the easiest and most consistent colours to detect through rigorous testing. This testing setup converts a camera image from PNG format to RGB888 HEX format via the Python script **im_to_tb.py** and then reads this image pixel-by-pixel in the SystemVerilog testbench **sim_id.sv**. Observing the waveforms, a simple Python script **joints_to_img.py** then plots small white boxes centered around the output marker center coordinates to verify that the correct locations are found. Going back to pixel detection, if the pixel is thus detected as red, green, or white, *pixel_is_colour* is set high for the next pipeline stage to process. Additional information such as the pixel's row and column on the image frame are also passed through.

4.2.2.2 Edge Detection

In the second stage of the pipeline, “Edge Detection”, the module first checks the current state of this stage; whether it is tracking a segment or not. If it is not tracking a segment, i.e. *in_segment* is low, then it checks whether *pixel_is_colour* is high to detect a possible “rising edge”. This means that the module will now start tracking the following pixels in case they form a segment of a marker together. On the other hand, if it is currently tracking a segment, there are four main cases taken into consideration. If a coloured pixel is encountered and detected correctly, it continues in tracking mode and if a “black” pixel is encountered and detected correctly, then the module should complete the segment and detect a “falling edge”. When a “falling edge” is detected, information such as the segment’s edges and computed midpoint are passed through to the next pipeline stage. Since the segment midpoint includes an 11-bit addition, the module made use of a DSP block to perform this computation.

While these cases cover an ideal image, there may exist external unidealities such as lighting and background in a real image, causing a pixel to be detected incorrectly. Therefore, denoising was also implemented in this pipeline stage to filter out mistakes, focusing on the case of a “black” pixel being detected incorrectly in the middle of a marker segment shown in Figure 7. The other case was not considered as the demonstration setup and strict colour thresholds ensured that it would not pose issues. This denoising was implemented by incrementing a counter, *gap_count*, to relax the constraints of a possible segment. Comparing the value of the counter to some parameter *MAX_GAP*, the module allows a certain number of consecutive “black” pixels to occur without setting the flag high for a “falling edge”.

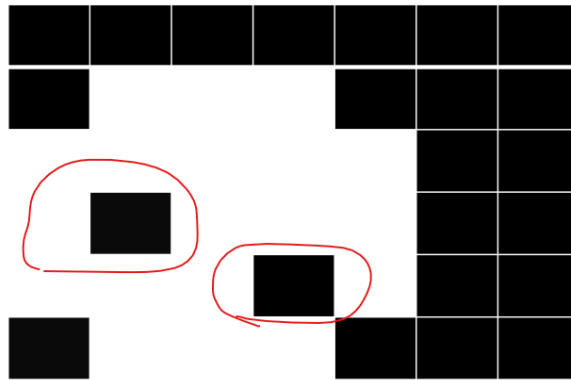


Figure 7. Example of “black” pixel noise.

4.2.2.3 Circle Tracking

In the third stage of the pipeline, “Circle Tracking”, the module first checks whether the previous stage detected a “falling edge”. There are two main cases to consider after a complete segment is detected. If the segment is part of a marker that is not being currently tracked in the *circles* array, then its information is stored and the module starts tracking it. On the other hand, if the segment is part of a marker that is already being tracked, then the edges and midpoints of this detected marker are updated accordingly. This comparison matching the segment to currently tracked markers is done in a combinational loop outside of the pipeline to minimize delay and prevent a possible bottleneck. Since the x and y midpoints are updated using a simple average, a DSP block was utilized to handle the 11-bit and 10-bit addition operations.

As mentioned previously, denoising is an important feature to consider when dealing with an unideal setup. If there are small portions of coloured segments in the background matching that of markers, it would confuse the module and incorrectly detect additional “circles”, or markers shown in Figure 8 below. In order to filter this noise, the module implements a replacement threshold. While it initially accepts any detected segment, a new segment would replace its spot in the *circles* array if it fails to meet a minimum height requirement, *MIN_HEIGHT*.

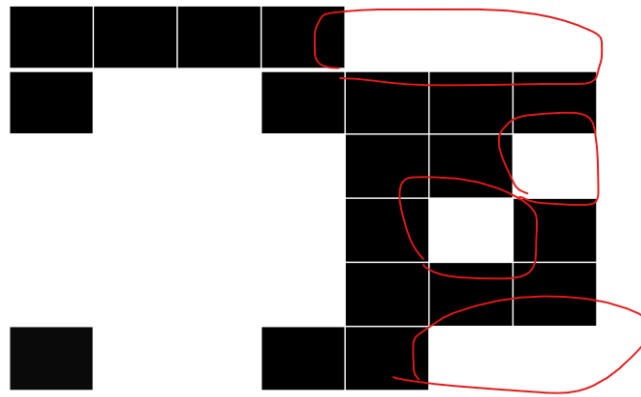


Figure 8. Example of “white” pixel noise.

Finally, the module also verifies the status of the flag, *stage_2_eof*, which indicates the end of the frame. By the next clock cycle, the x and y output coordinates of each marker center will be updated, and the output valid signal will be pulsed for one clock cycle. As explained in Section 4.2.1, the internal registers of the AXI4-lite slave interface will also then be updated accordingly for the Microblaze Processor to read. At the same time, the internal processing registers of the **joint_detect** module will also be reset, allowing the next frame to be processed. Since these registers are only used in the last pipeline stage, they will be reset by the earliest time the first pixel of the next frame could enter.

4.3 Microblaze Processor + AXI peripherals

The core control structure for our real-time 2D motion capture system involves the Microblaze Processor and its associated AXI peripherals. The Microblaze Processor is responsible for controlling the video pipeline and image processing, initializing relevant IPs and coordinating with the Custom IP. The AXI interconnect maps the processor to memory-mapped peripherals in our block design such as the VDMA, Custom IP, UART, and GPIO.

4.3.1 Microblaze Processor

The Microblaze Processor executes the embedded C application responsible for our system’s logic, visualisation, and user interaction. The primary functions are outlined below:

- **Custom IP Communication:** upon each new frame, the Microblaze Processor activates the Custom IP by writing “1” to one of its addresses. It then waits for the valid output interrupt from the Custom IP. Upon receiving this interrupt, the processor reads the coordinates of the 13 different markers from the slave registers of the Custom IP. These marker coordinates are then saved into a structure.

- **Color Detection:** when detecting colored markers, the processor needs to assign a color to each marker coordinate. It does this by sampling a 5x5 pixel square around the marker coordinate in the framebuffer and averaging the RGB values. Based on a threshold, the marker is classified as one of three colors: white, green, or red. The marker coordinates and associated colors are then saved into a structure.
- **Joint Assignments and Skeleton Drawing:** after the processor reads the marker coordinates, stores them, and classifies them if detecting colored markers, it then has to assign each coordinate to a specific joint. This classification is visualised in Figure 9.
 - **Monochrome Mode:** when detecting white markers only, the position of the joints are hardcoded based on location. The markers are first sorted in baskets based on their Y coordinate, with each basket representing a specific row. These baskets are then sorted based on X coordinate to determine the specific joint. For example, the first basket contains the marker with the highest Y coordinate, this marker is classified as the head. The next three highest Y coordinate markers make up the second basket, which are classified to the left shoulder, neck, and right shoulder based on their X coordinate. Hence, the skeleton is only accurately drawn if the relative position of the joints follow exactly what is shown in Figure 9.
 - **Color Mode:** with colored marker detection, this constraint is slightly relaxed. For the three different colors (white, red, and green), the upper row must never go below the lower row and for each row, the left must never go past the right. As an example, the two elbows must always remain above the two feet and the left elbow must always remain on the left of the right elbow. In this scenario, markers of the same color are sorted in the Y coordinate and then the X coordinate to classify the joints. This method allows for more freedom of movement, such as moving the hands above the head.
 - **Rendering:** after the joints are assigned, the skeleton is then drawn using hardcoded assignments, such as the head marker must be connected to the neck marker.

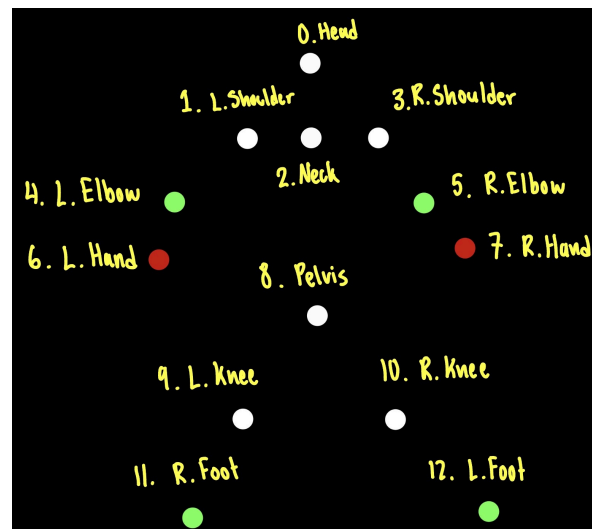


Figure 9. Skeleton joint classification visualisation. Note that this example is with colored markers, however, the colored markers are simply replaced by white markers for white marker only detection.

- **Line and Marker Drawing/Erasing:** the skeleton is drawn with 5x5 square joint markers connected by lines. The lines are drawn using Bresenham's Line Algorithm, which only uses simple operations such as addition, subtraction, and bit shifting [5]. To further optimize the algorithm, the pixel address pointer is manually shifted using addition after finding the starting address. Additionally, we record the history of skeleton markers and line locations to simplify erasing. As a result, we can simply draw black pixels over these specific locations rather than setting the entire frame to black.
- **GUI:** the processor manages an on-screen graphical user interface (GUI) controlled by the buttons on the Video board. It handles state transitions between video mode and photo mode in addition to toggling between white and colored marker detection. Text is drawn using a 7-segment character font, including a FPS counter calculated using an AXI timer. The GUI is shown in video mode in Figure 10a) and photo mode in Figure 10b).
- **Photo Mode:** in photo mode, the video stream is paused when the middle button is pressed. The frozen frame is sent to the custom IP and then the skeleton is drawn. Pressing the middle button again erases the previous skeleton and resumes the video stream.
- **Video Mode:** in video mode, the skeleton is drawn over a black background in real-time. To eliminate flickering, double buffering is used by displaying one framebuffer and drawing on the off-screen framebuffer. Once a frame is completely drawn, the processor waits for a V-sync (MM2S) interrupt from the VDMA before displaying the completed frame.

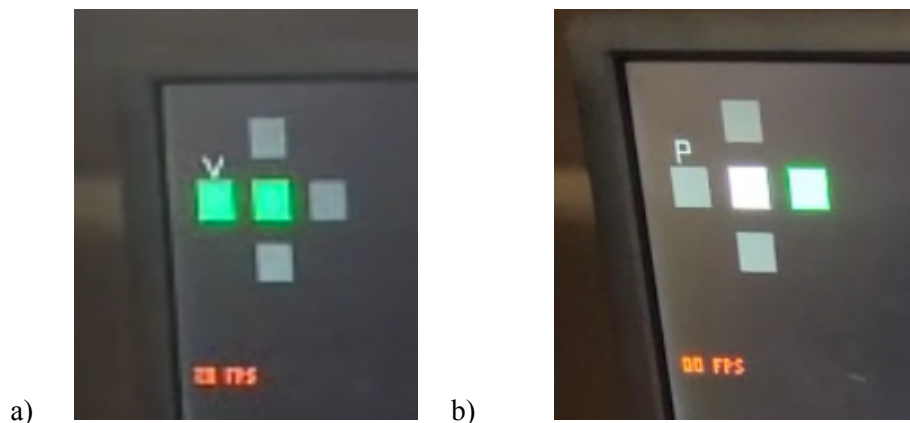


Figure 10. Picture of on-screen GUI in a) video mode and b) photo mode.

4.3.2 AXI UART

As a byproduct of the HDMI demo provided by Diligent, we used the AXI UART peripheral for user interaction and displaying debugging outputs to an external terminal. It receives input commands from the user to update system parameters, such as resolution and framebuffer index. Additionally, it outputs useful debugging information such as the location and color assignment of joints.

4.3.3 AXI GPIO

The AXI GPIO module links the software and the physical hardware on the Video board. Two primary GPIO instances are used:

- **Video Pipeline Control:** one GPIO module is used to manage the initialization and reset states of the video capture hardware components.

- **D-Pad Inputs:** a second GPIO module is mapped to the D-pad on the Video board, with the buttons continuously being polled to interact with the system and the GUI. The top button exits the GUI, the left and right buttons switch between video and photo mode, the center button is an action button that acts as a pause/capture photo trigger, and the bottom button toggles between white and colored marker detection.

5. Design Tree Description/Video Link

Our design is posted online at the following Github link: [Github Repo Link](#)

The 2 minute video explaining and demonstrating the project is available to view at the following Google Drive link: [Video](#)

The design tree shown in Figure 11 lists the location and description of the files that can be found in the repository. These include:

- Custom IP Verilog code, including testbenches used to simulate the Custom IP for real images.
- Software: the Vitis 2024.1 project is included, along with a standalone C file that contains all of the code for the Microblaze Processor.
- Hardware: the Vivado 2024.1 project that was used can be recreated using the tcl script and the list of custom IPs.
- Final report and presentation PDF.
- Link to the 2 minute video explaining and demonstrating the project.

```
Real-time-2D-Motion-Capture-System/
├── Custom IP Verilog/                # List of Verilog files for the Custom IP
│   ├── axi_slave_lite.v             # AXI interface
│   ├── custom_ip_top.v              # Custom IP top module
│   ├── joint_detect.v               # Marker detection
│   └── test/                         # Testbenches used for the Custom IP
│       ├── img_to_tb.py
│       ├── input.png
│       ├── joints_to_img.py
│       └── sim_id.sv
├── Microblaze C Code/                # C code for Microblaze Processor
│   └── full_video_532.c
├── Vitis Classic 2024.1 Project/      # Vitis Classic 2024.1 Software Project File
│   └── Group9_SW.zip
├── Vivado 2024.1 Project/            # Vivado 2024.1 Hardware Project Files (can rebuild the project using these files)
│   ├── IP_Repo/                     # IP Folder
│   ├── Nexys-Video-HW1/
│   ├── marker_detector_ip/          # Custom IP source code folder
│   └── Nexys-Video-HW.tcl           # .tcl file used to rebuild the project
├── Final_Presentation.pdf            # Final presentation PDF
├── Final_Report_Group_9.pdf          # Final report PDF
└── README.md
```

Figure 11. Design tree for the project Github repository.

6. Advice for Future Students

- When working with colour detection, we would recommend using HSV instead of RGB. This is because during our project, depending on the physical conditions and lighting, RGB had difficulty distinguishing between different colours.
- When working with camera inputs, the OV5640 camera pins are not compatible with the PMOD headers on the Nexys Video Board.
- Plan on writing testbenches and integrating modules into your design as soon as possible. Many issues will come up during testing and integration, and modules rarely work on the first implementation.
- Having a detailed high-level overview of your design is helpful before developing individual modules. It allows the team to understand what components are and are not needed, and to identify logical errors.
- Have alternatives to certain risk prone modules inside your system in case they don't work out and you can switch over to them if needed.

References

- [1] Digilent, Inc., “Nexys Video™ FPGA Board Reference Manual,” rev. Sept. 1, 2020.
- [2] Digilent, Inc., “Nexys Video HDMI Demo,” Digilent Reference. [Online]. Available: <https://digilent.com/reference/programmable-logic/nexys-video/demos/hdmi>
- [3] Digilent, Inc., “RGB-to-DVI (Source) 1.4 IP Core User Guide,” rev. Sept. 6, 2017. [Online]. Available: <https://github.com/Digilent/vivado-library/blob/master/ip/rgb2dvi/docs/rgb2dvi.pdf>
- [4] B. Vanca, “Re: Can I change the reference clock for Dynamic Clock Generator?,” Digilent Forum, Jul. 26, 2018. [Online]. Available: <https://forum.digilent.com/topic/15573-can-i-change-the-reference-clock-for-dynamic-clock-generator/>
- [5] Jack E. Bresenham, “Algorithm for computer control of a digital plotter,” *IBM Systems Journal*, vol. 4, no. 1, pp. 25–30, 1965, doi: 10.1147/sj.41.0025.